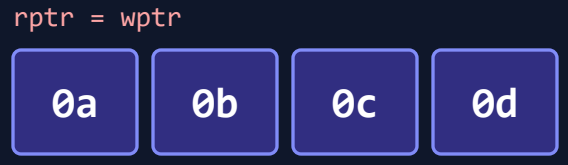
VERILOG · HOMEWORK

FIFO

설계 · 시뮬레이션 검증



DEPTH 4 × 8 bit · push → wptr, pop → rptr

정안성

구조

control_unit + register_file

wptr · rptr · full · empty

읽기

조합 출력 (FWFT)

pop_data = mem[rptr]

TB

directed 4 + random 256

compare_buffer 모델 비교

결과

fail 0

random pop 113 / 113 PASS

Contents

DEPTH 4 FIFO
push · pop · 경계 상태 검증

00 **Summary** — 핵심 수치 · 시나리오 결과 — p. 3

01 **목적** — FIFO 설계 목표 · 검증 항목 — p. 4

02 **Reference** — FIFO 개념 · register_file vs BRAM — p. 5 – 6

03 **블록 다이어그램** — fifo 내부 (control_unit · register_file) — p. 7

04 **동작 설계** — control_unit 동작표 · 예상 타이밍 — p. 8 – 9

05 **시뮬레이션** — TB 구조 · 시나리오 · S1 ~ S5 — p. 10 – 17

06 **트러블 슈팅** — 검증 중 확인한 경계 동작 — p. 18

07 **결론 · 고찰** — 결론 · 고찰 · 개선 방향 — p. 19

A **APPENDIX** — TB 로그 — p. 21

00

DEPTH 4 FIFO 구현 · 검증 결과

결론 · control_unit (wptr · rptr · full · empty) + register_file (4 × 8 bit, 조합 읽기)로 FIFO 구성
· directed 시나리오 4개 + random 256 iteration fail 0 — full · empty · 동시 push/pop 경계 포함

FIFO 크기

4 × 8 bit

DEPTH 4 · pointer 2 bit

읽기 방식

FWFT

pop_data = mem[rptr] 조합 출력

directed 판정

11 / 11

S1 ~ S4 pass 메시지

random pop 비교

113 / 113

256 iteration · fail 0

동시 push+pop

5 · 8

full 중 5회 · empty 중 8회 (random)

wptr 순환

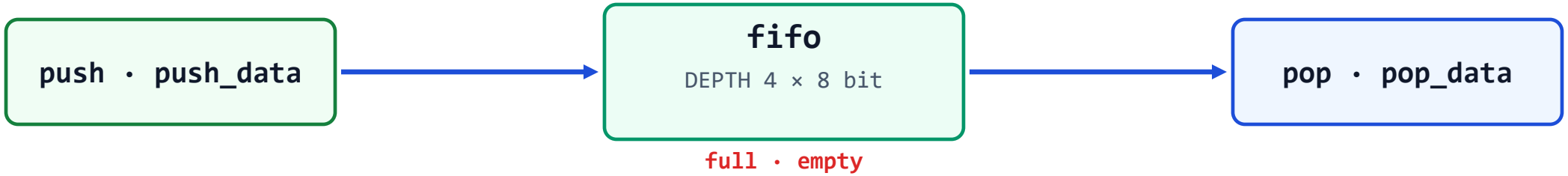
29

random 구간 3 → 0 횟수

01

Push · Pop 순서를 지키고 경계 상태에서 오동작 없는 FIFO

결론
- **FIFO = 먼저 넣은 byte를 먼저 꺼내는 버퍼 → UART 수신 · 송신 사이 속도 차 흡수용**
- **full · empty · 동시 push/pop 경계에서도 순서 · 개수가 깨지지 않는지 TB로 확인**



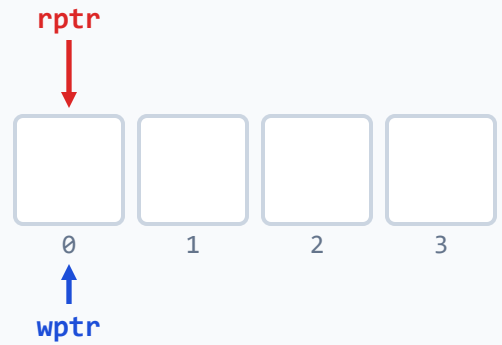
#	검증 항목	목표 동작	결과
S1	**Reset**	reset 해제 후 full 0 · empty 1	2 / 2
S2	**Push × 5**	4번째 push 후 full 1 · 5번째 push 무시	파형 확인
S3	**Pop × 5**	넣은 순서 (0a → 0d)로 출력 · 4번째 pop 후 empty 1	5 / 5
S4	**Push + Pop × 5**	empty 중 push만 · 이후 개수 유지하며 순서대로 출력	4 / 4
S5	**Random 256**	무작위 push · pop에서 compare_buffer와 불일치 0	113 / 113

02

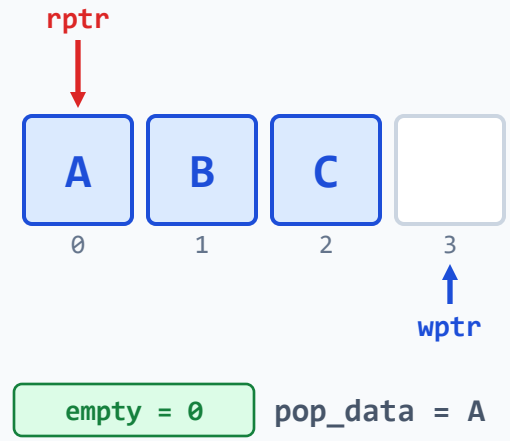
FIFO: wptr · rptr 두 포인터로 순서를 지키는 원형 버퍼

결론
- **push = wptr 자리에 쓰고 wptr + 1, pop = rptr + 1 → 두 포인터 비교로 full · empty 판단**
- **pop_data = mem[rptr] → pop 전에도 맨 앞 byte가 출력에 보임 (FWFT)**

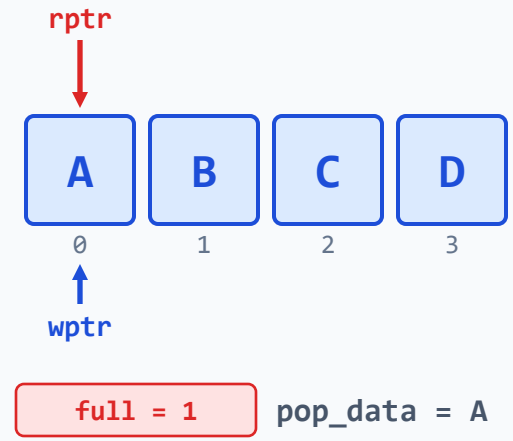
1 reset



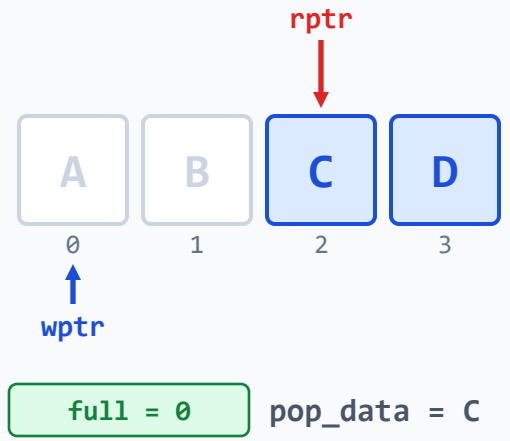
2 push A · B · C



3 push D



4 pop × 2



1 **full** push 후 wptr이 rptr을 따라잡음 → full ← 1

2 **empty** pop 후 rptr이 wptr을 따라잡음 → empty ← 1

3 **wptr = rptr** empty · full 둘 다 같음 → flag로 구분

4 **FWFT** 읽기 = 조합 출력 → TX가 바로 맨 앞 byte 사용

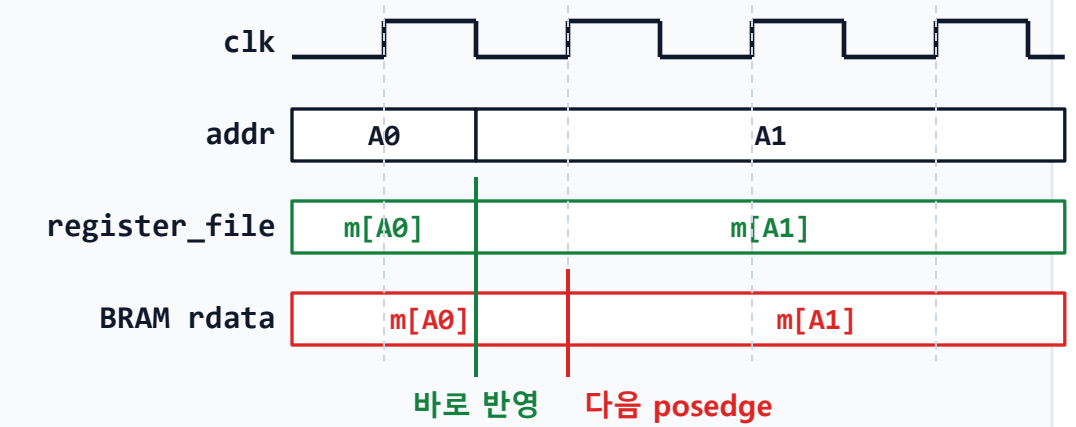
02

저장 공간: register_file은 조합 읽기, BRAM은 1 clk 뒤 읽기

결론
- **fifo의 register_file은 rdata = mem[raddr] 조합 출력 → pop 전에도 맨 앞 byte가 보임 (FWFT)**
- **BRAM_IP는 posedge에 rdata 등록 → 주소를 바꾼 다음 clk에 데이터 → FIFO에 쓸 경우 읽기 1 clk 지연 고려**

항목	register_file (fifo.sv)	BRAM_IP (BRAM.v)
크기	4 × 8 bit	256 × 8 bit
쓰기	posedge · we = push & ~full	posedge · we = 1
읽기	조합: rdata = mem[raddr]	등록: we = 0일 때 posedge에 rdata ← ram[addr]
읽기 지연	0 clk (주소 변경 즉시)	1 clk
FIFO 적용	pop_data가 곧 맨 앞 byte	pop 뒤 1 clk 기다려 데이터 사용

읽기 타이밍 비교 (개념도)



점선 = posedge

1 **FWFT** empty = 0인 동안 pop_data = 맨 앞 byte
TB는 negedge에 pop_data를 바로 비교

2 **BRAM FIFO** 깊이를 키울 경우 BRAM 사용 가능
대신 pop 후 1 clk 지연 handshake 필요

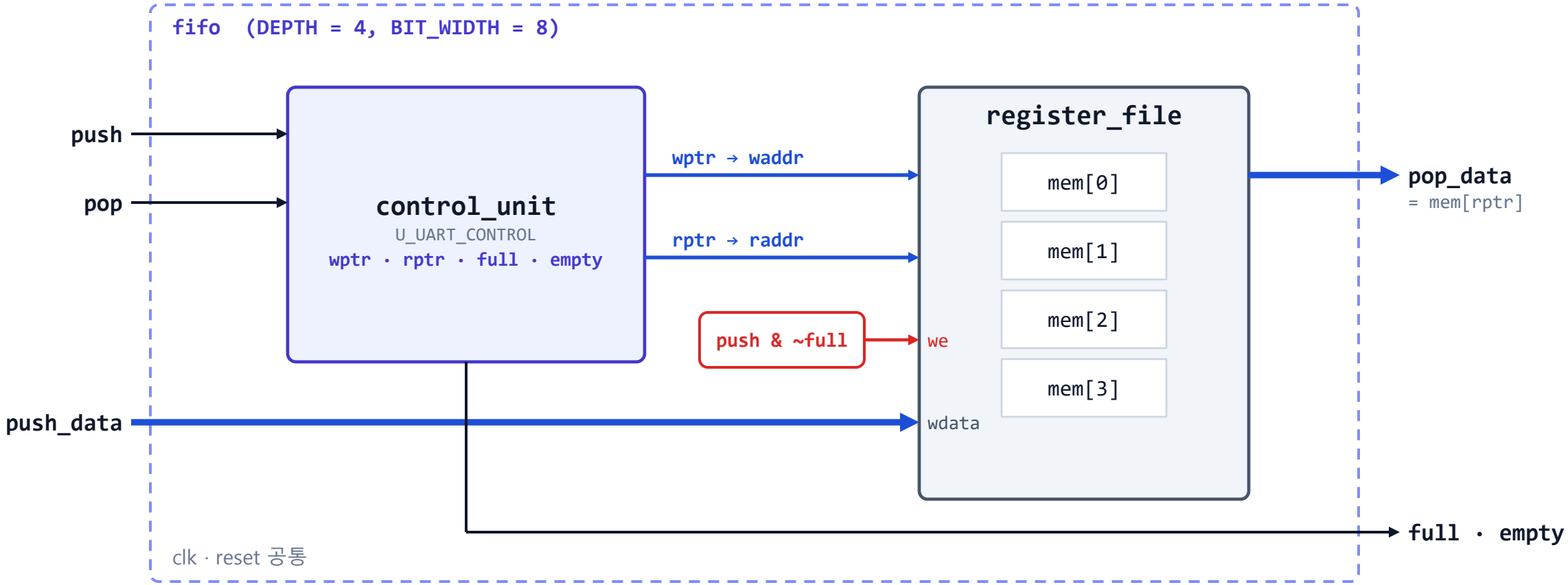
3 **tb_bram** 03_RAM/sim_file/tb_bram.v: addr 10 · 11 · 31 · 32에 쓰기 → 같은 주소 읽기 (BRAM 읽기 1 clk 지연 확인용)

03

fifo = control_unit (포인터 · flag) + register_file (4 × 8 bit)

결론

- **control_unit: {push, pop}와 full · empty로 wptr · rptr · flag 갱신**
- **register_file: we = push & ~full일 때만 쓰기, 읽기 pop_data = mem[rptr] 조합 출력**



1 **control_unit** {push, pop} 조합별 포인터 · flag 갱신 (다음 장)

2 **register_file** 쓰기는 clk, 읽기는 조합 → FWFT

04

{push, pop} × empty · 중간 · full 동작

결론 · **full일 때 push 무시 · empty일 때 pop 무시**
· **push + pop 동시 — empty: push만 · 중간: 둘 다 · full: pop만 (구조상 spec)**

입력 / 상태	empty	중간 (1 ~ 3개)	full
push	wptr + 1 empty ← 0	wptr + 1 wptr = rptr일 경우 full ← 1	무시 (we = 0)
pop	무시	rptr + 1 rptr = wptr일 경우 empty ← 1	rptr + 1 full ← 0
push + pop	push만 wptr + 1 · empty ← 0	wptr + 1 · rptr + 1 개수 유지	**pop만 (spec)** **rptr + 1 · full ← 0**

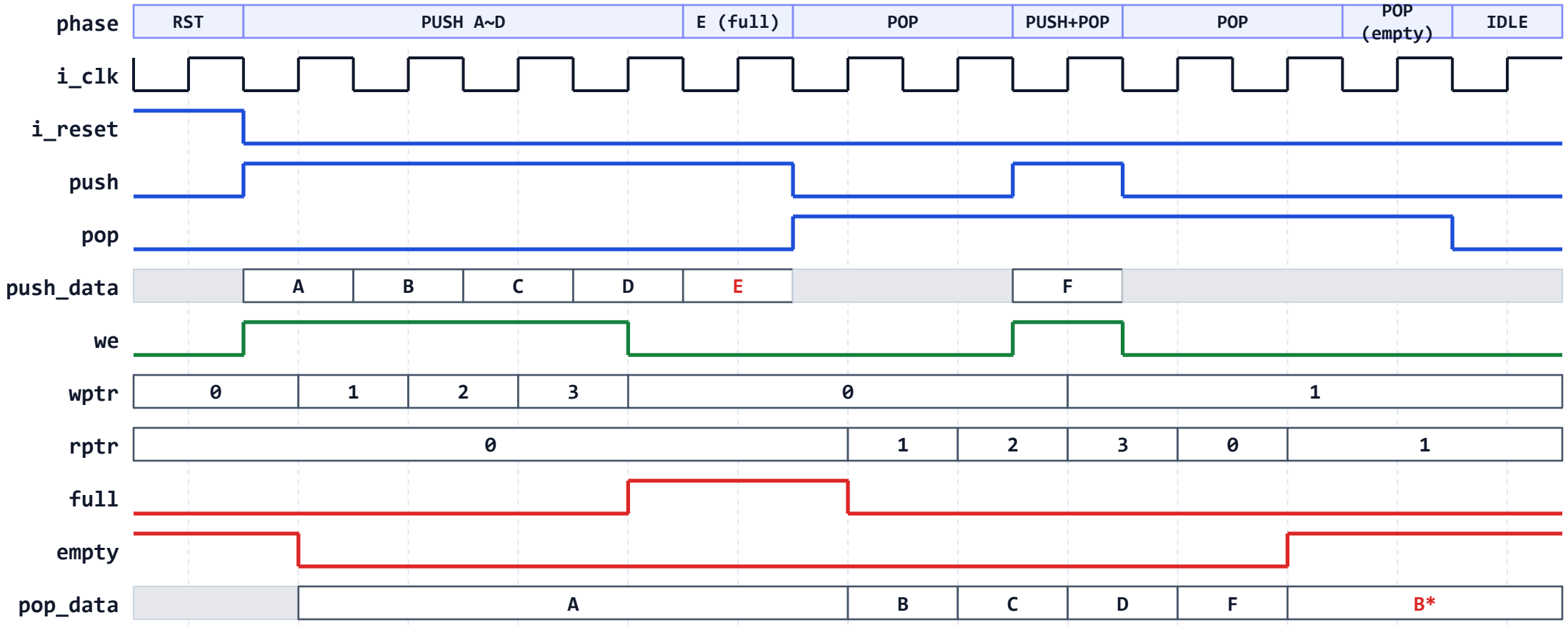
1 **full + push·pop** we = push & ~full → 쓰기 차단 · pop만
그 clk 입력 byte는 버려짐 (spec)

2 **TB 검증** S2 ~ S5에서 경계 경우를 compare_buffer와 비교
→ random pop 113 / 113 PASS

FIFO 예상 타이밍: 입력은 Negedge, 레지스터는 Posedge에서 갱신

04

결론 · push · pop은 negedge에 바꾸고 posedge에서 wptr · rptr · full · empty 갱신 → TB도 같은 규칙
· full 중 push (E)는 we = 0으로 무시, empty 뒤 pop_data는 이전 값 유지 (B*)

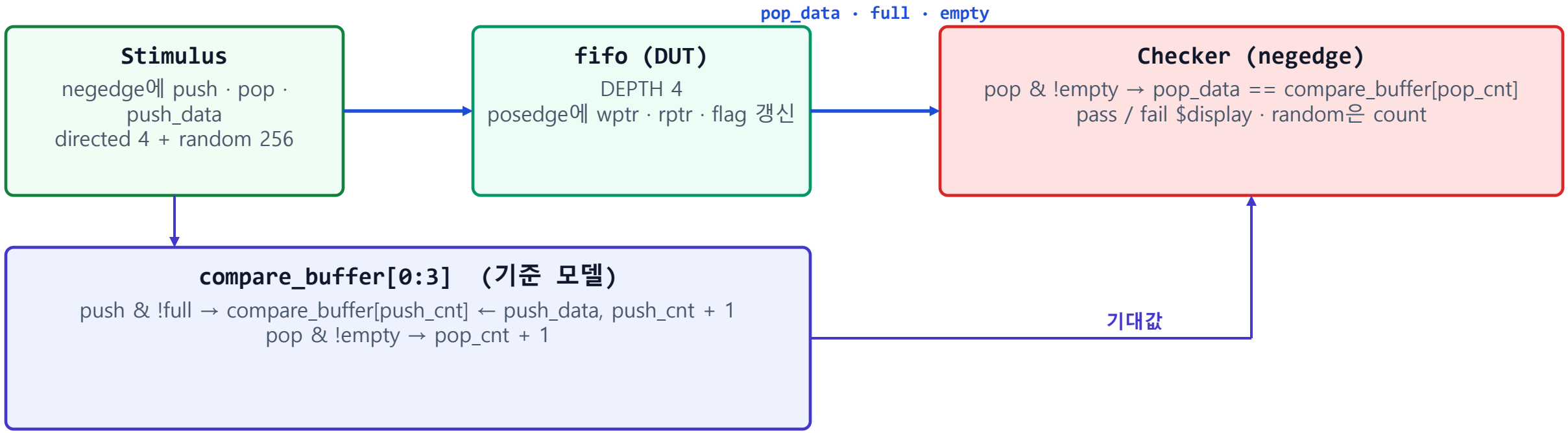


점선 = posedge · 출처: 03_RAM/sim_file/fifo_wavedrom.txt (설계 의도)

05

TB: Negedge 구동 · Negedge 판정, compare_buffer가 기준 모델

결론
- **push 값은 full = 0일 때만 compare_buffer에 기록, pop은 empty = 0일 때만 pop_data와 비교**
- **push_cnt · pop_cnt 2 bit → 4칸에서 자동 순환 (DUT의 wptr · rptr과 같은 방식)**



1. **판정 시점** negedge = posedge 반 clk 전
   → 그 순간 pop_data (FWFT 맨 앞) 비교

2. **full / empty** flag는 판정 시점 값 사용
   → full 중 push는 모델에도 기록 안 함

3. **결과 출력** directed: 시점별 pass / fail 메시지 · random: pass count · fail count 집계 ($display)

FIFO 검증 시나리오 5개 모두 기대값과 일치

05

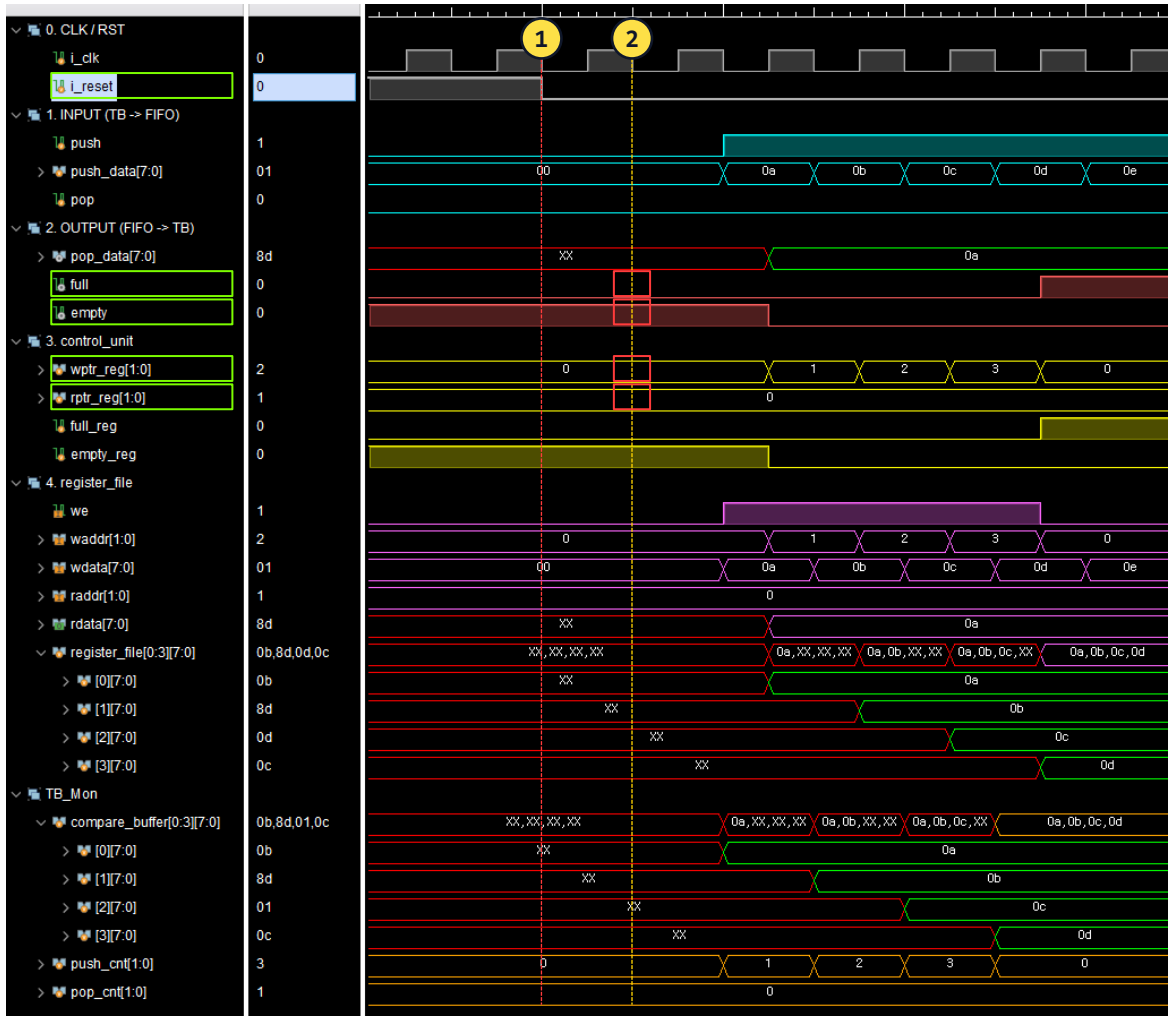
결론 · directed 4개로 reset · full · empty · 동시 push/pop 확인 → random 256으로 섞인 순서 확인
· 시각은 xsim 기준 (clk 10 ns, 입력은 negedge)

#	시나리오	입력 (Stimulus)	확인 신호	예상 결과	결과
S1	**Reset**	i_reset 1 → 0 (20 ns)	full · empty (30 ns)	full 0 · empty 1	2 / 2
S2	**Push × 5**	0a ~ 0e push (40 ~ 90 ns)	wptr · full · register_file	4번째 push 후 full 1 · 0e 무시	파형 확인
S3	**Pop × 5**	pop 5회 (100 ~ 150 ns)	pop_data · rptr · empty	0a → 0d 순서 · empty 1	5 / 5
S4	**Push + Pop × 5**	0f ~ 0b push + pop (160 ~ 210 ns)	pop_data · wptr · rptr · empty	첫 clk push만 · 이후 개수 1 유지	4 / 4
S5	**Random 256**	push · pop · data 무작위 (240 ~ 2800 ns)	pop_data vs compare_buffer	불일치 0	113 / 113

05

S1 Reset: 해제 직후 full 0 · empty 1

결론 · i_reset 1 → 0 (20 ns) 뒤 첫 negedge (30 ns)에 flag 확인
· full 0 · empty 1 · wptr = rptr = 0 → PASS 2 / 2



① 20 ns i_reset 1 → 0

② 30 ns negedge 판정

full 0 · empty 1

wptr 0 · rptr 0 → PASS

PASS
2 / 2
full · empty 판정

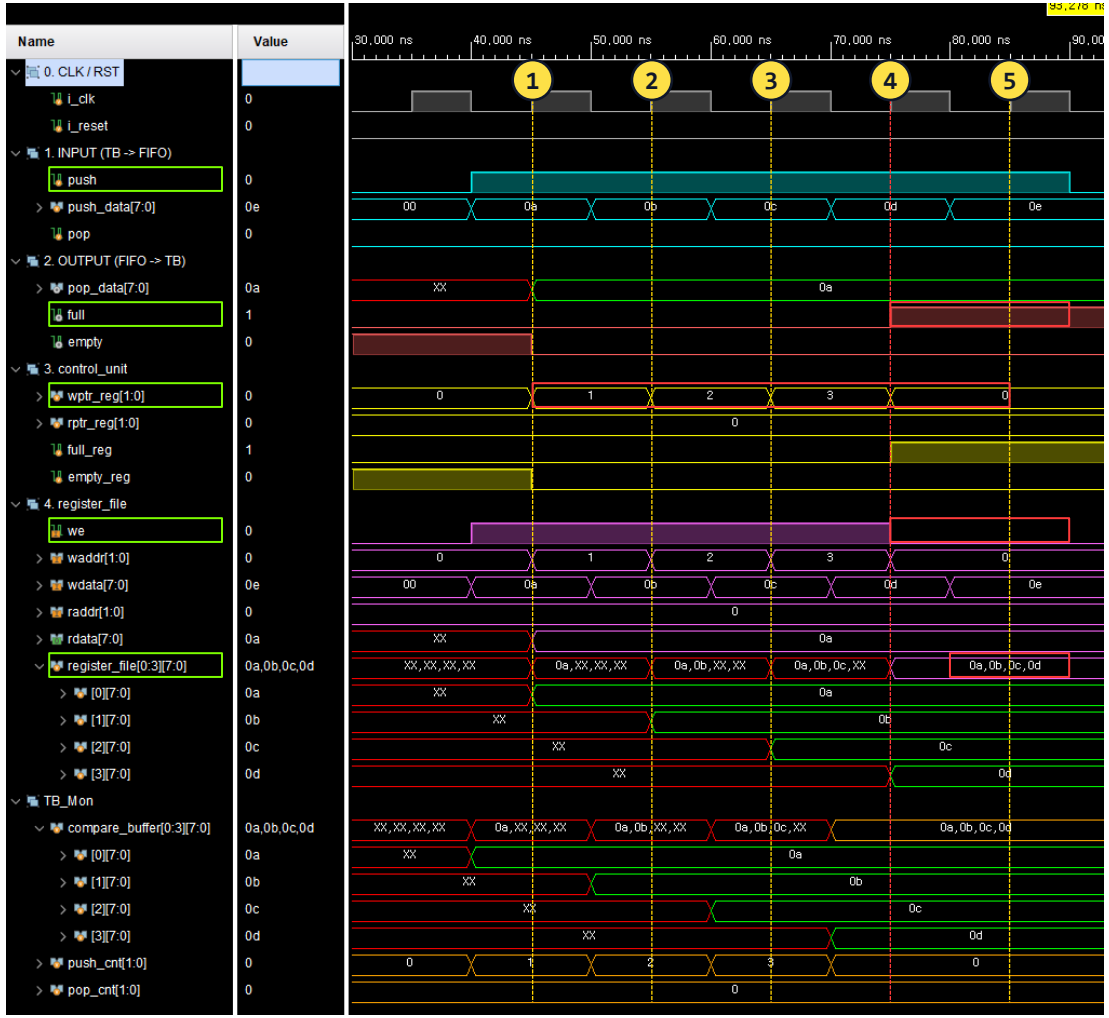
xsim 측정 (30 ns)

full	0
empty	1
wptr / rptr	0 / 0

S2 Push × 5: 4번째 Push 후 Full, 5번째는 무시

05

결론
- 0a ~ 0d push → posedge마다 wptr 0 → 1 → 2 → 3 → 0, 4번째 뒤 (75 ns) full 1
- full 중 0e push → we = 0 → wptr · register_file 그대로 (모델에도 기록 안 함)



1. 45 ns 0a → wptr 1
2. 55 ns 0b → wptr 2
3. 65 ns 0c → wptr 3
4. 75 ns 0d → wptr 0, full 1
5. 85 ns 0e → we 0, 무시

PASS
파형 확인
TB 자동 판정 없음

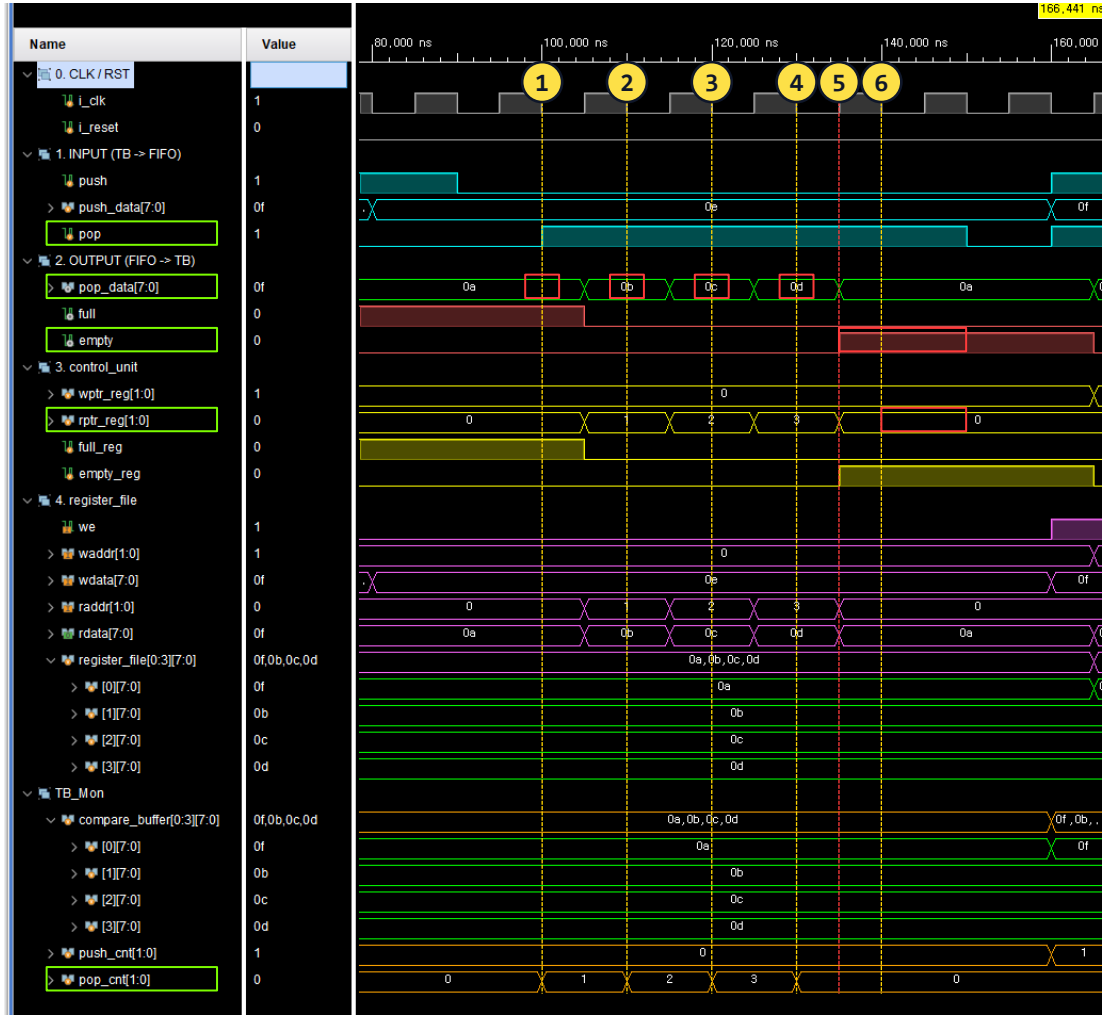
xsim 측정

wptr 갱신	**45 ~ 75 ns**
full 1	**75 ns**
0e push (80 ns)	**무시**
register_file	**0a 0b 0c 0d**

S3 Pop × 5: 넣은 순서대로 0a → 0d, 4번째 Pop 후 Empty

05

결론 · negedge마다 pop_data를 compare_buffer와 비교 → 0a · 0b · 0c · 0d 순서 일치
· 4번째 pop 뒤 (135 ns) empty 1 → 5번째 pop은 무시 (rptr 그대로)



1. 100 ns pop_data 0a 판정
2. 110 ns 0b 판정
3. 120 ns 0c 판정
4. 130 ns 0d 판정
5. 135 ns empty 1
6. 140 ns pop → 무시

PASS
5 / 5
pop 4 + empty 1

xsim 측정

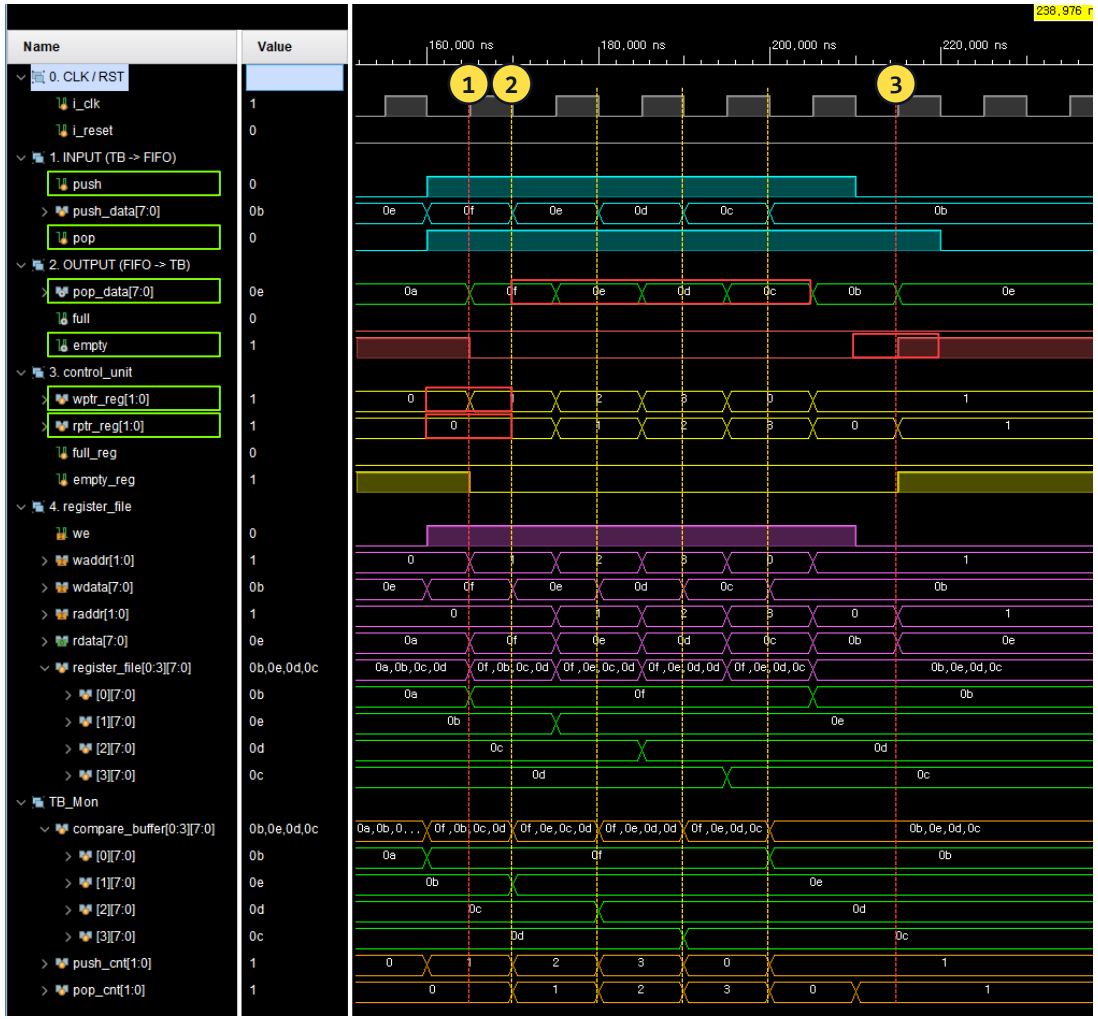
pop_data 비교	**100 ~ 130 ns**
순서	**0a 0b 0c 0d**
empty 1	**135 ns**
5번째 pop (140 ns)	**무시**

S4 Push + Pop × 5: Empty 중엔 Push만, 이후 개수 1 유지

05

결론

- 160 ns empty 중 push + pop → push만 (0f 저장, empty 0)
- 170 ~ 200 ns push + pop → wptr · rptr 같이 +1, pop_data 0f · 0e · 0d · 0c 순서 일치



1. 165 ns push만: wptr +1, empty 0
2. 170 ~ 200 ns 0f 0e 0d 0c 판정

   wptr · rptr 같이 +1 (개수 1)
3. 215 ns pop → empty 1

PASS

4 / 4

pop_data 판정

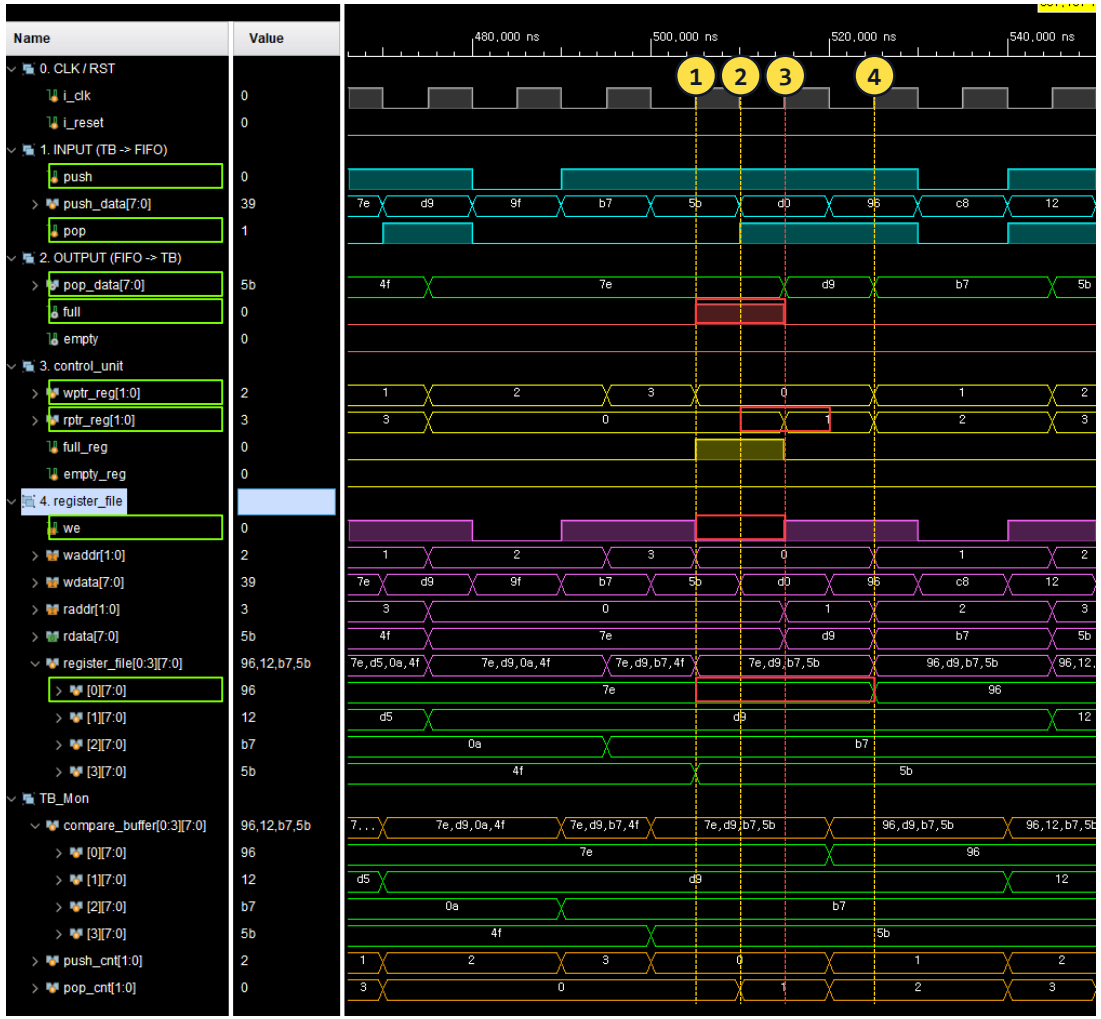
xsim 측정

160 ns	**push만 (0f)**
pop_data	**0f 0e 0d 0c**
개수	**1 유지**
210 ns pop	**→ empty 1**

05

S5-A Random: Full 중 Push+Pop은 Pop만 (Spec)

결론 · 505 ns full 1 → 510 ns push+pop (d0) → rptr만 +1, full 0 · we 0이라 d0 저장 안 함
· pop_data 7e 판정 PASS · 모델도 d0 기록 안 함 → 이후 순서 일치 (random 113 / 113)



1. 505 ns 5b push → full 1
2. 510 ns push+pop (d0) · 7e 판정
3. 515 ns rptr만 +1, full 0 · we 0 → d0 버림
4. 525 ns push+pop 96 → 둘 다 +1

PASS
113 / 113
pop 비교 · fail 0

경계 경우 (xsim 측정)

push & full	**5**
pop & empty	**7**
push+pop & empty	**8**
push+pop & mid	**55**
push+pop & full	**5**
wptr 순환	**29**

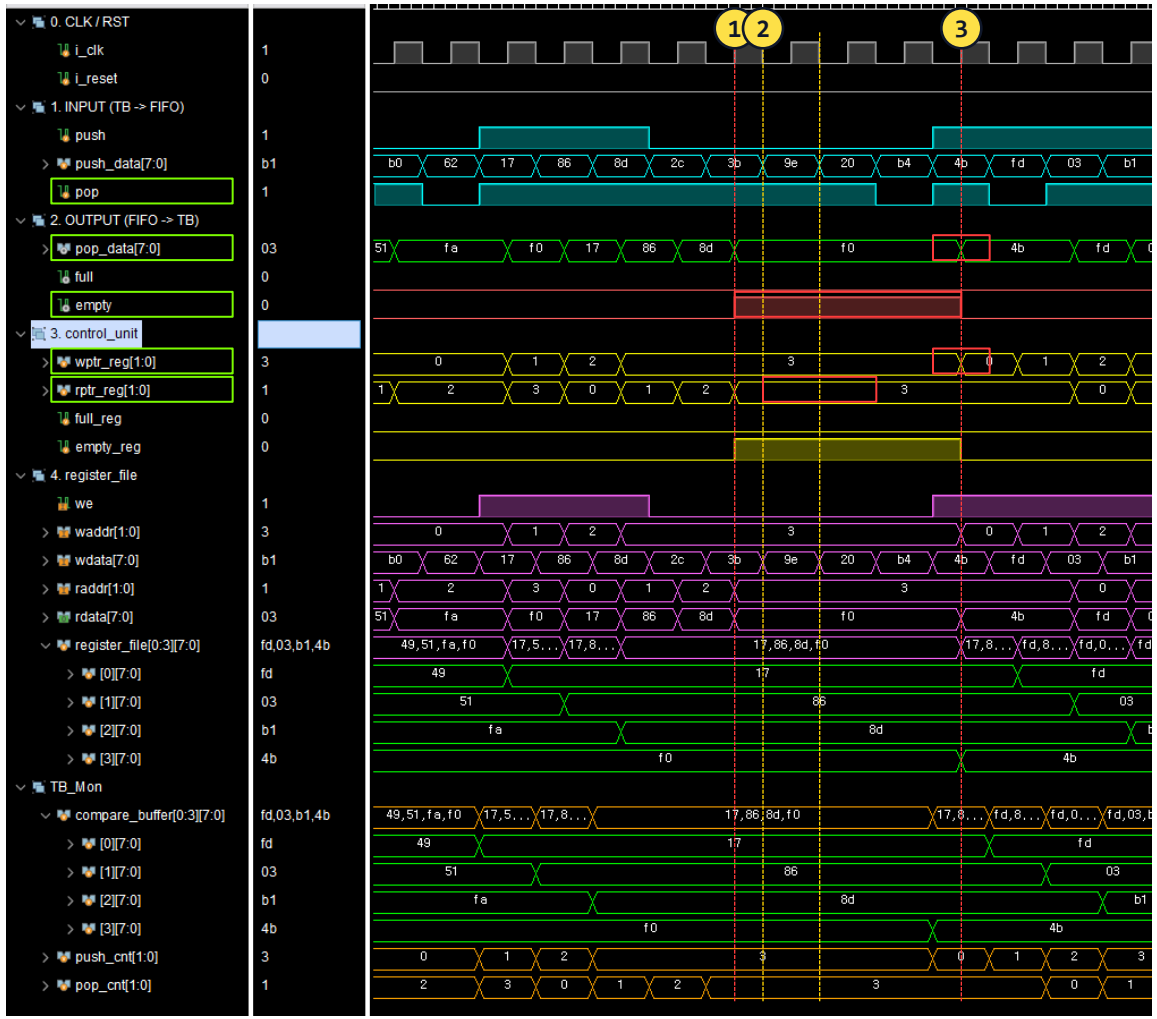
random 256 iteration (240 ~ 2790 ns) 기준

05

S5-B Random: Empty 중 Pop 무시, Push+Pop은 Push만

결론

- **1405 ns empty 1 → 1410 · 1420 ns pop 무시 (rptr 3 유지)**
- **1440 ns push+pop (4b) → wptr만 +1, empty 0 → 다음 clk부터 pop_data 4b**



1. 1405 ns pop → empty 1
2. 1410 · 1420 ns pop → 무시 (rptr 3)
3. 1445 ns push+pop (4b) → push만

   wptr 3 → 0 · empty 0 · pop_data 4b

PASS

113 / 113

pop 비교 · fail 0

경계 경우 (xsim 측정)

push & full	**5**
pop & empty	**7**
push+pop & empty	**8**
push+pop & mid	**55**
push+pop & full	**5**
wptr 순환	**29**

random 256 iteration (240 ~ 2790 ns) 기준

검증 중 확인한 경계 동작 3가지

06

결론 · 시나리오 5개 모두 PASS → FAIL로 이어진 문제 없음
· 구조상 동작 3가지 확인 — full 중 push 무시 · full 중 push+pop은 pop만 · empty 중 push+pop은 push만

1 full 중 push → 무시

- we = push & ~full → 쓰기 차단
- wptr · flag 그대로
- TB 모델도 full = 1일 때 기록 안 함

실제 발생

```
S2  80 ns  0e
S5  700 · 1560 · 1570
    2140 · 2150 ns
```

2 full 중 push+pop → pop만

- control_unit: full 상태 → rptr만 +1, full ← 0
- 그 clk의 push_data는 버려짐
- 모듈 구조상 정한 동작

실제 발생

```
S5  510 ns  d0 버림
    640 · 900 · 1290
    2640 ns
```

3 empty 중 push+pop → push만

- empty 상태 → pop 무시, wptr만 +1
- empty ← 0, 다음 clk부터 pop 가능
- pop_data는 그 다음 clk에 새 값

실제 발생

```
S4  160 ns  0f
S5  250 · 1000 · 1140
    1440 ns 외 4회
```

07

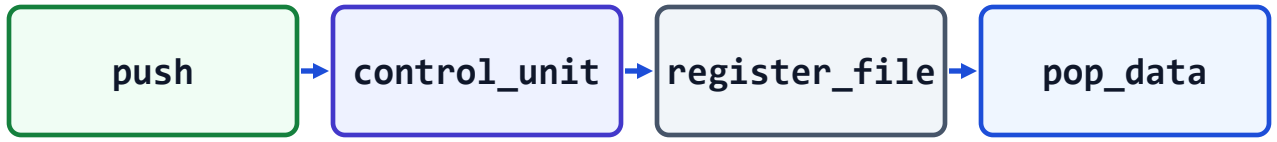
FIFO 순서 · 경계 동작 모두 기대값과 일치

결론 · directed 4개 (reset · full · empty · 동시 push/pop) + random 256 → fail 0
· full 중 push+pop은 pop만 하는 구조상 spec까지 TB 모델과 일치

결론

시나리오	결과	판정
S1 Reset	full 0 · empty 1	PASS
S2 Push × 5	full 1 · 0e 무시 (파형)	PASS
S3 Pop × 5	0a → 0d · empty 1	PASS
S4 Push + Pop × 5	push만 → 개수 1 유지	PASS
S5 Random 256	113 / 113	PASS

구조 요약



고찰

- S2 full 확인은 TB 자동 판정이 없어 파형으로만 확인
- directed 결과는 메시지만 출력, 개수 집계는 random만
- $random seed 고정 → 매번 같은 순서의 random

개선 방향

1. directed 단계도 pass / fail count + full · empty 자동 판정
2. seed를 바꿔 random 반복 · 경계 경우 횟수 출력
3. 깊이를 키울 경우 BRAM 기반 FIFO (읽기 1 clk 지연 handshake)

Q & A

감사합니다

정안성

A

TB 로그: Directed 11 PASS · Random 113 PASS

결론 · Tcl console 출력 (directed pass 메시지 + random result)
· random result: total 256 · pass 113 · fail 0

directed (S1 ~ S4) · pass 메시지

```
 30000 :pass full signal
 30000 :pass empty signal
100000 : pass pop data
110000 : pass pop data
120000 : pass pop data
130000 : pass pop data
140000 : pass empty signal
170000 : pass pop data
180000 : pass pop data
190000 : pass pop data
200000 : pass pop data
```

random (S5) · result

```
*** random result ***
total random test  =       256
pass count =       113
fail count =         0
```